If you are the CTO

You see everything, including the notes. That reach is logged where the person it is about can read it — which is what makes it acceptable rather than quietly corrosive.

ihelp-ops-ai-decodeds-projects.vercel.app

Nobody types how far along they are.

There is no “80% done” box anywhere in this platform, and there never will be. A task moves because something real happened: a branch appeared, a commit landed, a pull request opened, a person approved it, it merged.

You cannot fall behind by forgetting to update a status. If you did the work, the platform already knows.

You also cannot get ahead by saying you did. Nobody can, including the founder.

Your day

One person. The only role that can read a 1:1 it did not write.

Morning — The digest is in your inbox from 6pm yesterday: who moved, who was quiet, who is on leave. Read the quiet list

	as a prompt for a conversation, never as a verdict — plenty of days are reading, debugging or meetings.
Analytics	Cycle time and review wait are the two that describe the system rather than the people. If review wait is climbing, reviews are the bottleneck and no amount of pushing on authors will help.
Unblocking	Assign or reassign anything, in any repository. Approve leave for anybody. Merge is available to you, and can still be refused by branch protection — that refusal is protection working.
People	1:1 notes and feedback. Nothing you write about somebody is hidden from them; that is enforced by Postgres, not by a guideline. Every note you read that you did not write appears in their access log.
Accounts	**Admin** is where roles, agent tiers and GitHub logins are set. Setting somebody's GitHub login is the thing that turns an account into a person who can own work.

What you can do

Generated from the same file the platform enforces, so it cannot describe a permission you do not have.

Sign in and read the board, team and analytics	`yes`
Book and withdraw your own leave	`yes`
Take an unassigned task	`with GitHub linked`
Start a task, open a pull request, merge, comment	`with GitHub linked`
Dispatch agents	`up to your tier`

Talk to any agent about a task	`yes`
Read somebody else's conversation with an agent	no
Assign work to other people, and reassign	`yes`
Approve or reject leave	`everyone`
Read somebody else's leave and balance	`everyone`
Read a 1:1 you did not write	`all, and it is logged`
Read feedback written about somebody else	`yes`
See who was nudged, and local editor sessions	`everyone`
Change roles, tiers, pods and GitHub logins	`yes`

Worth knowing

> **You cannot read anybody's agent conversations.** The policy has one clause and no admin escape, and `npm run test:chat` proves it by signing in as a founder and failing to read one. If that ever needs to change, it is a schema change with a written reason, not a setting.

> **You cannot change your own row.** Nobody can, including you. A role change should be something a second person agreed to.

> **The last admin is protected.** Nothing can demote or deactivate the only remaining admin — the way back would be a hand-written SQL statement against production.

> **Every role change is recorded.** Who, when, and what it was before. The person it was about reads it on their own page, and nobody can edit or remove a line.

Talking to an agent

Every task page has a conversation. Four things about it are worth knowing before you use it.

It can read, not write	The agent sees the task, its comments and its own brief from the repository, and will read the code with you, review an approach, or say plainly that it does not know. It cannot edit a file, commit or open a pull request from the conversation. When the work needs doing, press **Run** — that dispatches the same agent into GitHub Actions, where it opens a pull request somebody reviews
It is private, and it is not the record	Nobody else can read your conversations — not your lead, not the CTO, not the founder. Nothing said in one moves a task or counts as work. If something from it matters, put it on the issue as a comment. That is the record; this is the thinking
Any agent, from your first day	Tiers decide which agents you may *dispatch*, because a dispatched agent changes code. Talking changes nothing, so you may talk to all eight — including the architect
It costs money	Each answer shows what it cost, usually a fraction of a cent. It is charged to the company like any agent run

How a task moves

Seven steps, each caused by a real thing.

10%	The issue was opened — it may have no owner yet

20%	A branch was created for it
40%	The first commit landed on that branch
60%	A pull request was opened
75%	The automated checks passed
90%	A human approved it
100%	It was merged

A task stuck at 10% means nothing has happened — not that somebody forgot to update it.

If something looks wrong

Everything says 10%	No GitHub activity has reached the platform. The board says so rather than showing a quiet week
Merge was refused	Branch protection. A required review or check is missing — working as intended
My commits aren't showing	The commit hook isn't installed in that clone. Run `git config core.hooksPath .githooks` and push again
There's no Take it button	Either the task already has an owner — it says who — or your GitHub account isn't linked
The same number twice	Different tasks in different repositories. Issue numbers restart in each one

Every error in this platform says what failed and what to check. One that doesn't is worth reporting.

The handbook covering everything, for everybody, is the one to read once. This one is the part that is yours.

Questions about the platform go to the founder. Questions about a task go on the task.